

CSE232 시스템프로그래밍(캡스톤디자인)

5주차 두 번째 수업

# 시간과 시계 경과 시간과 달력 시간

경민기

# 왜 시간을 다루는가

---

로그: 사건에 타임스탬프를 남겨 나중에 순서를 재구성한다.

측정: 코드가 얼마나 오래 걸리는지 초·밀리초로 잰다.

판정: 타임아웃·만료·유효 기간을 시각과 비교한다.

정렬: 파일과 기록을 시각 순서로 세운다.

지난 시간에 잠깐 본 CLK\_TCK도 오늘 이야기로 이어집니다.

# 1부

## 달력 시간

time\_t와 struct tm으로 시각을 다룹니다.

# time(2)과 time\_t

---

## 에포크

1970-01-01 00:00:00 UTC부터  
지난 초 단위 정수다.

## time(&t) / time(NULL)

현재 시각의 time\_t를 얻는다.  
실패는 (time\_t)-1이다.

## difftime(3)

두 time\_t의 차를 초 단위  
실수로 돌려준다.

헤더: <time.h>. 초 단위 해상도입니다.

# calendar\_demo: 시각 조회와 출력

```
time_t now = time(NULL);
if (now == (time_t)-1) { perror("time"); return 1; }

struct tm *tm = localtime(&now);
if (tm == NULL) { perror("localtime"); return 1; }

char buf[64];
size_t n = strftime(buf, sizeof buf,
                    "%Y-%m-%d %H:%M:%S %Z", tm);
if (n == 0) /* 버퍼가 작으면 0 */
    return 1;
printf("local=%s\n", buf);
```

## 조회와 변환

time으로 초를 얻고  
localtime으로 분해한다.

## 문자열화

strftime이 tm을 사람이  
읽는 시각으로 만든다.

## 반환값

time은 -1, localtime은 NULL,  
strftime은 0이 실패다.

# date(1) 명령과 비교

```
$ date +%s
1790493335
$ date
2026. 09. 27. (일) 16:15:35 KST
$ date -u
Sun Sep 27 07:15:35 UTC 2026

# TZ 환경변수로 시간대를 바꿔 본다
$ TZ=Asia/Seoul date
2026. 09. 27. (일) 16:15:35 KST
$ TZ=UTC date
Sun Sep 27 07:15:35 UTC 2026
```

출력 예시: 형식은 로컬에 따라 다릅니다.

## date +%s

에포크 초를 그대로 출력한다.  
calendar\_demo의 epoch과 같다.

## TZ 환경변수

프로세스의 시간대를 정한다.  
localtime이 이 값을 본다.

## 교차 확인

같은 시각을 두 문으로 얻어  
형식과 시간대를 확인한다.

# struct tm과 변환 함수

---

## localtime / gmtime

time\_t를 지역·UTC 기준  
struct tm으로 분해한다.

## mktime

struct tm을 time\_t로 되돌린다.  
범위 밖 값은 정규화된다.

## tm\_mon은 0부터

0이 1월, 11이 12월이다.  
tm\_year는 1900년 기준 오프셋.

반환: struct tm 포인터, 실패 NULL.

# 실습 1: 달력 시간 관찰

```
TZ=Asia/Seoul ./calendar_demo
TZ=UTC ./calendar_demo

./tm_normalize

# 같은 시각을 다른 문으로 확인
date; date -u; date +%s
```

값을 예측한 뒤 REPORT.md에 기록합니다.

## 시간대 차이

Seoul과 UTC의 local 문자열이 정확히 9시간 차이인지 본다.

## 정규화

1월 32일·2월 29일이 mktime 뒤 어떻게 바뀌는지 적는다.

## 교차 확인

epoch 값이 date +%s와 같은 초인지 확인한다.

# 코드 읽기 · calendar\_demo.c (1/6)

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <time.h>
  → L1-L3: perror·printf·시간 함수 헤더. 시간 API는 모두 <time.h>에 있다.
4
5 /* 달력 시간의 기본 흐름: time(2)으로 초를 얻고
6    localtime·gmtime으로 struct tm을 만들어 strftime으로 문자열로 바꾼다. */
  → 주석: 이 파일의 호출 흐름(조회→분해→문자열)을 먼저 적었다.
7 int main(int argc, char **argv)
  → main. 이 프로그램은 명령행 인자를 받지 않는다.
8 {
9     if (argc != 1) {
10         fprintf(stderr, "usage: %s\n", argv[0]);
11         return 2;
12     }
  → L9-L12: 인자가 있으면 usage를 남기고 종료 상태 2. 관례상 2는 사용법 오류다.
```

## 코드 읽기 · calendar\_demo.c (2/6)

```
13     time_t now = time(NULL);  
→ time(NULL): 현재 시각을 에포크 초 time_t로 얻는다.  
14     if (now == (time_t)-1) {  
15         perror("time");  
16         return 1;  
17     }  
→ L14-L17: 실패는 (time_t)-1. perror가 errno를 문장으로 바꿔 준다.  
18     printf("epoch=%lld\n", (long long)now);  
→ time_t의 크기는 시스템마다 다르므로 long long으로 바꿔 %lld로 찍는다.  
19  
20     char buf[64];  
→ 시각 문자열 버퍼. 64바이트는 이 형식에 넉넉하다.  
21     struct tm *tm_local = localtime(&now);  
→ localtime: time_t를 지역 시간대 기준 struct tm으로 분해한다.  
22     if (tm_local == NULL) {  
23         perror("localtime");
```

## 코드 읽기 · calendar\_demo.c (3/6)

```
24     return 1;
25 }
→ L22-L25: localtime 실패는 NULL. 구조체를 채운 뒤에만 포인터를 읽는다.
26 if (strptime(buf, sizeof buf, "%Y-%m-%d %H:%M:%S %Z", tm_local) == 0) {
27     fprintf(stderr, "strptime: buffer too small\n");
28     return 1;
29 }
→ L26-L29: strptime 변환 실패는 0, 곧 버퍼 부족이다.
30 printf("local=%s\n", buf);
→ 완성된 지역 시각 문자열을 출력한다.
31
32 struct tm *tm_utc = gmtime(&now);
→ gmtime: 같은 분해를 UTC 기준으로 수행한다.
33 if (tm_utc == NULL) {
34     perror("gmtime");
35     return 1;
```

## 코드 읽기 · calendar\_demo.c (4/6)

```
36     }
    → L33-L36: gmtime 실패도 NULL로 검사한다.
37     if (strftime(buf, sizeof buf, "%Y-%m-%d %H:%M:%S UTC", tm_utc) == 0) {
38         fprintf(stderr, "strftime: buffer too small\n");
39         return 1;
40     }
    → L37-L40: UTC 문자열도 같은 방식으로 만든다. %Z 대신 고정 표기를 붙인다.
41     printf("utc=%s\n", buf);
    → UTC 시각 문자열 출력. TZ와 무관하다.
42
43     /* difftime은 time_t끼리의 차를 초 단위 실수로 돌려준다.
44        time(2)는 초 단위 해상도이므로 1.2초를 기다려 차이를 만든다. */
    → 주석: difftime 구간. time(2)는 초 단위 해상도라 1.2초를 기다린다.
45     struct timespec start, tick;
    → 대기 시작을 단조 시계로 기록한다.
46     if (clock_gettime(CLOCK_MONOTONIC, &start) == -1) {
```

## 코드 읽기 · calendar\_demo.c (5/6)

```
47     perror("clock_gettime");
48     return 1;
49 }
→ L46-L49: clock_gettime 실패는 -1. MONOTONIC은 거꾸로 가지 않는다.
50 do {
→ do-while: 조건을 뒤에서 검사하므로 판독을 최소 한 번은 한다.
51     if (clock_gettime(CLOCK_MONOTONIC, &tick) == -1) {
52         perror("clock_gettime");
53         return 1;
54     }
→ L51-L54: 현재 판독을 다시 얻는다. 차는 같은 시계끼리만 낸다.
55     } while ((tick.tv_sec - start.tv_sec) * 1000000000L
56             + (tick.tv_nsec - start.tv_nsec) < 12000000000L);
→ L55-L56: 나노초로 환산한 차가 1.2초에 닿으면 탈출한다.
57     time_t later = time(NULL);
→ 두 번째 time 판독. 첫 판독과 초 단위로 비교된다.
```

## 코드 읽기 · calendar\_demo.c (6/6)

```
58     if (later == (time_t)-1) {
59         perror("time");
60         return 1;
61     }
    → L58-L61: 조회 실패 경로. L14-L17과 같은 계약이다.
62     printf("difftime=%.1fs\n", difftime(later, now));
    → difftime: 두 time_t의 차를 초 단위 실수로 돌려준다.
63     if (fflush(stdout) == EOF) {
64         perror("stdout");
65         return 1;
66     }
    → L63-L66: 출력 버퍼를 밀고 실패면 종료 상태 1. 관례상 1은 실행 실패다.
67     return 0;
    → 정상 종료 상태 0.
68 }
```

# 코드 읽기 · tm\_normalize.c (1/5)

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <time.h>
   → L1-L4: memset을 위해 string.h, 시간 함수를 위해 time.h를 포함한다.
5
6 /* struct tm을 직접 채워 mktime으로 정규화한다.
7    tm_mon은 0부터 센다: 0이 1월, 11이 12월이다. */
   → 주석: 이 프로그램이 관찰하는 함정(tm_mon 0 기반)을 먼저 적었다.
8 static int normalize(const char *label, int year, int mon, int mday)
   → 정규화 실험 하나를 수행하는 함수. 라벨과 연·월·일을 받는다.
9 {
10     struct tm tm_in;
11     memset(&tm_in, 0, sizeof tm_in);
   → L10-L11: struct tm을 만들고 memset으로 0을 채워 쓰레기 값을 막는다.
12     tm_in.tm_year = year - 1900;
   → tm_year는 1900년 기준 오프셋이다. 2026년은 126을 넣는다.
```

## 코드 읽기 · tm\_normalize.c (2/5)

```
13     tm_in.tm_mon = mon;
    → tm_mon은 0부터 센다. 인자를 그대로 넣어 함정을 드러낸다.
14     tm_in.tm_mday = mday;
    → tm_mday는 1부터 센다. 32처럼 범위 밖 값도 그대로 넣는다.
15     tm_in.tm_hour = 12;
16     tm_in.tm_min = 0;
17     tm_in.tm_sec = 0;
    → L15-L17: 시·분·초를 정오로 고정한다. 자정 경계를 피하기 위해서다.
18     tm_in.tm_isdst = -1; /* 시스템이 서머타임 여부를 정하게 한다 */
    → tm_isdst=-1: 서머타임 여부를 mktime이 판단하게 맡긴다.
19
20     printf("%s: %04d-%02d-%02d 입력 (tm_mon=%d)\n",
21           label, year, mon + 1, mday, mon);
    → L20-L21: 입력을 먼저 찍는다. mon+1이 사람이 아는 월 번호다.
22     time_t t = mktime(&tm_in);
    → mktime: tm을 time_t로 바꾸며 범위 밖 필드를 정규화한다.
```

## 코드 읽기 · tm\_normalize.c (3/5)

```
23     if (t == (time_t)-1) {
24         perror("mktime");
25         return 1;
26     }
    → L23-L26: 표현할 수 없는 시각이면 (time_t)-1로 실패한다.
27     char buf[32];
28     if (strftime(buf, sizeof buf, "%Y-%m-%d %H:%M:%S", &tm_in) == 0) {
29         fprintf(stderr, "strftime: buffer too small\n");
30         return 1;
31     }
    → L27-L31: 정규화된 tm을 strftime으로 문자열로 만든다.
32     static const char *const wday[] =
33         {"일", "월", "화", "수", "목", "금", "토"};
    → L32-L33: 요일 이름표. tm_wday는 0이 일요일이다.
34     printf("  mktime 결과 %s (%s요일, tm_wday=%d)\n",
35           buf, wday[tm_in.tm_wday], tm_in.tm_wday);
    → L34-L35: 결과와 요일을 출력한다. mktime이 tm_wday까지 채웠다.
```

# 코드 읽기 · tm\_normalize.c (4/5)

```
36     return 0;
    → 이 함수의 성공은 0이다.
37 }
38
39 int main(int argc, char **argv)
    → main. 실험 네 개를 차례로 호출한다.
40 {
41     if (argc != 1) {
42         fprintf(stderr, "usage: %s\n", argv[0]);
43         return 2;
44     }
    → L41-L44: 인자 오류는 usage와 종료 상태 2.
45     int status = 0;
    → 실험별 실패를 모아 종료 상태에 반영한다.
46     status |= normalize("1월 32일", 2026, 0, 32);
    → 1월 32일: 2월 1일이 되는지 본다. tm_mon=0이 1월이다.
```

# 코드 읽기 · tm\_normalize.c (5/5)

```
47     status |= normalize("12월 32일", 2026, 11, 32);  
    → 12월 32일: 다음 해 1월 1일로 넘어간다. 연도 넘김도 정규화가 처리한다.  
48     status |= normalize("2025-02-29", 2025, 1, 29);    /* 평년 2월 29일 */  
    → 평년 2025-02-29는 3월 1일이 된다.  
49     status |= normalize("2024-02-29", 2024, 1, 29);    /* 윤년 2월 29일 */  
    → 윤년 2024-02-29는 그대로 살아 있다.  
50     if (fflush(stdout) == EOF) {  
51         perror("stdout");  
52         return 1;  
53     }  
    → L50-L53: stdout flush를 확인한다.  
54     return status;  
    → 한 번이라도 실패했으면 0이 아닌 상태가 된다.  
55 }
```

## 2부

# 경과 시간

clock\_gettime과 단조 시계로 흐름을 잡니다.

# clock\_gettime(2)과 struct timespec

---

## struct timespec

tv\_sec와 tv\_nsec 필드로  
초와 나노초를 함께 담는다.

## 클럭 아이디

CLOCK\_REALTIME,  
CLOCK\_MONOTONIC 등을 고른다.

## 반환값

성공 0, 실패 -1이다.  
없는 시계는 EINVAL이다.

헤더: <time.h>. CLOCK\_\* 상수도 같은 헤더입니다.

# 두 시계: REALTIME과 MONOTONIC

---

CLOCK\_REALTIME: 벽시계다. 달력 시각의 기준이고 NTP가 조금씩 수정한다.

CLOCK\_MONOTONIC: 단조 시계다. 거꾸로 가지 않는다.

REALTIME은 과거로 점프할 수 있다: 측정에 쓰면 음수 간격이 나온다.

MONOTONIC의 기준점은 부팅 이후라 시각 표시에는 못 쓴다.

측정은 MONOTONIC, 표시는 REALTIME이 오늘의 원칙이다.

# timespec 차이 계산

```
static long long ts_ns(const struct timespec *ts)
{
    return (long long)ts->tv_sec * 1000000000LL
        + ts->tv_nsec;
}

long long start = now_ns(CLOCK_MONOTONIC);
/* ... 재려는 코드 ... */
long long elapsed =
    now_ns(CLOCK_MONOTONIC) - start;
printf("%lldns\n", elapsed);
```

완성 예제: elapsed\_demo.c의 ts\_ns

## 정수 환산

초·나노초를 나노초  
정수 하나로 합친다.

## 정수 뺄셈

double 없이 빼므로  
오차가 생기지 않는다.

## 표시 환산

밀리초는 1e6으로 나눠  
표시할 때만 바꾼다.

# elapsed\_demo: 측정 흐름

---

## 시작·끝 기록

작업 앞뒤로 MONOTONIC을  
읽어 차를 구한다.

## 반복 측정

5번 재고 min·median·max로  
노이즈를 가른다.

## 워밍업

첫 측정 전에 한 번 돌려  
캐시 효과를 줄인다.

완성 예제: elapsed\_demo.c

# 정밀도와 함정

---

해상도와 정밀도는 다르다: 나노초 필드가 있어도 실제 오차는 더 크다.

측정 자체도 시간을 쓴다: `clock_gettime` 호출 비용이 결과에 섞인다.

계산은 정수 나노초로, 나눗셈은 표시할 때만 한다.

`clock_getres`로 시계의 해상도를 조회할 수 있다.

한 번의 측정보다 여러 번의 최솟값이 안정적이다.

# 반복 측정과 노이즈

---

## 한 번 측정

스케줄·캐시·인터럽트가  
값을 흔든다.

## 다섯 번 측정

min은 최상 조건,  
median은 전형적인 값이다.

## 비교 기준

개선 전후는 같은 조건의  
같은 통계값끼리 비교한다.

## 실습 2: 경과 시간 측정

```
./elapsed_demo
./elapsed_demo --clocks
./clock_show

# 나노초를 밀리초로 환산
python3 -c 'print(3492000/1e6, "ms")'
```

값을 예측한 뒤 REPORT.md에 기록합니다.

### 측정 값

run1..run5의 ns 값과  
min·median을 적는다.

### 시계 비교

--clocks로 REALTIME과  
MONOTONIC의 now·res를 본다.

### 기준 차이

REALTIME now는 에포크 초,  
MONOTONIC now는 부팅 후 초다.

# 코드 읽기 · elapsed\_demo.c (1/9)

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <time.h>
   → L1-L5: qsort를 위해 stdlib.h가 필요하다.
6
7 /* timespec을 나노초 정수로 바꾸고 빼기를 정수로 처리한다. */
   → 주석: 이 파일의 계약, timespec을 나노초 정수로 다룬다.
8 static long long ts_ns(const struct timespec *ts)
   → timespec을 나노초 정수 하나로 바꾸는 도우미 함수.
9 {
10     return (long long)ts->tv_sec * 1000000000LL + ts->tv_nsec;
   → 초에 10^9를 곱해 나노초를 더한다. long long으로 오버플로를 피한다.
11 }
12
13 static long long now_ns(clockid_t clock)
   → 지정한 시계의 현재값을 나노초로 돌려주는 래퍼.
```

## 코드 읽기 · elapsed\_demo.c (2/9)

```
14 {
15     struct timespec ts;
16     → 판독 결과를 담는 지역 timespec.
17     if (clock_gettime(clock, &ts) == -1) {
18         perror("clock_gettime");
19         exit(1);
20     }
21     → L16-L19: clock_gettime 실패는 -1. 조희 프로그램이므로 바로 종료한다.
22     return ts_ns(&ts);
23     → 성공하면 나노초 값을 돌려준다.
24 }
25 static int cmp_ll(const void *a, const void *b)
26     → qsort가 쓸 비교 함수, long long 두 개의 대소 판정.
27 {
28     long long x = *(const long long *)a, y = *(const long long *)b;
29     → void 포인터를 long long으로 되돌려 읽는다.
```

## 코드 읽기 · elapsed\_demo.c (3/9)

```
26     return x < y ? -1 : x > y;  
    → x<y면 -1, x>y면 1, 같으면 0을 돌려준다.  
27 }  
28  
29 /* 재지정 불가 작업량: volatile 배열을 여러 번 훑는다. */  
    → 주석: 작업량을 재지정 불가능하게 만드는 이유를 적었다.  
30 static long long workload(void)  
    → 측정 대상 작업 함수.  
31 {  
32     static volatile int a[1000000];  
    → static volatile 배열. volatile이라 컴파일러가 반복을 지우지 못한다.  
33     long long sum = 0;  
    → 누산기. 결과를 반환해 최적화를 막는다.  
34     for (int pass = 0; pass < 4; ++pass)  
35         for (int i = 0; i < 1000000; ++i)  
36             sum += a[i] + i;  
    → L34-L36: 100만 개 배열을 네 번 훑는 밀리초 안팎의 작업이다.
```

## 코드 읽기 · elapsed\_demo.c (4/9)

```
37     return sum;
    → 합을 돌려준다.
38 }
39
40 static void spin_ns(long long ns)
    → 약속한 나노초만큼 바쁘게 기다리는 도우미.
41 {
42     long long start = now_ns(CLOCK_MONOTONIC);
    → 시작 판독.
43     while (now_ns(CLOCK_MONOTONIC) - start < ns)
44         ;
    → L43-L44: 경과가 목표에 닿을 때까지 빈 몸통 반복을 돈다.
45 }
46
47 static void show_clock(const char *name, clockid_t clock)
    → 시계 하나를 조사해 한 줄 출력하는 함수.
48 {
```

## 코드 읽기 · elapsed\_demo.c (5/9)

```
49     struct timespec now, res;
    → 현재값과 해상도를 담은 timespec 두 개.
50     if (clock_gettime(clock, &now) == -1) {
51         printf("%-24s unsupported (%s)\n", name, strerror(errno));
52         return;
    → L50-L52: 값 조회 실패는 미지원. 표시하고 계속한다(0 반환).
53     }
54     if (clock_getres(clock, &res) == -1) {
55         perror("clock_getres");
56         exit(1);
57     }
    → L54-L57: 해상도 조회 실패는 이 프로그램에서 치명적으로 보고 종료 1.
58     printf("%-24s now=%lld.%09ld res=%ld.%09ld\n", name,
59           (long long)now.tv_sec, now.tv_nsec,
60           (long)res.tv_sec, res.tv_nsec);
    → L58-L60: 이름·now·res를 한 줄로 찍는다. %-24s가 표의 열을 맞춘다.
61 }
```

## 코드 읽기 · elapsed\_demo.c (6/9)

```
62
63 int main(int argc, char **argv)
    → main. 측정 모드와 --clocks 모드를 나눈다.
64 {
65     int clocks_only = argc == 2 && strcmp(argv[1], "--clocks") == 0;
    → --clocks가 인자로 정확히 하나 왔는지 기록한다.
66     if (!(argc == 1 || clocks_only)) {
67         fprintf(stderr, "usage: %s [--clocks]\n", argv[0]);
68         return 2;
69     }
    → L66-L69: 그 외 인자 조합은 usage와 종료 상태 2.
70     if (clocks_only) {
    → --clocks 모드 진입.
71         show_clock("CLOCK_REALTIME", CLOCK_REALTIME);
    → 표준 벽시계를 찍는다.
72         show_clock("CLOCK_MONOTONIC", CLOCK_MONOTONIC);
    → 표준 단조 시계를 찍는다.
```

## 코드 읽기 · elapsed\_demo.c (7/9)

```
73 #ifdef CLOCK_MONOTONIC_RAW
74     show_clock("CLOCK_MONOTONIC_RAW", CLOCK_MONOTONIC_RAW);
75 #endif
    → L73-L75: RAW 시계는 매크로가 정의되어 있을 때만 컴파일된다.
76 #ifdef CLOCK_PROCESS_CPUTIME_ID
77     show_clock("CLOCK_PROCESS_CPUTIME_ID", CLOCK_PROCESS_CPUTIME_ID);
78 #endif
    → L76-L78: 프로세스 CPU 시계도 조건부 컴파일이다.
79     if (fflush(stdout) == EOF) {
80         perror("stdout");
81         return 1;
82     }
83     return 0;
    → L79-L83: flush를 확인하고 종료 상태 0으로 끝낸다.
84 }
85
86 enum { RUNS = 5 };
    → 측정 횟수 5를 enum 상수로 고정한다.
```

## 코드 읽기 · elapsed\_demo.c (8/9)

```
87     long long ns[RUNS];  
    → 측정값 배열.  
88     long long sink = 0;  
    → 최적화 방지용 누산기.  
89     spin_ns(2000000); /* 워밍업: 첫 측정의 캐시 효과를 줄인다 */  
    → 2ms 워밍업, 첫 측정의 캐시·페이지 효과를 줄인다.  
90     for (int i = 0; i < RUNS; ++i) {  
    → 측정 반복문.  
91         long long start = now_ns(CLOCK_MONOTONIC);  
    → 시작 판독.  
92         sink += workload();  
    → 작업 실행. 반환값을 sink에 더해 지워지지 않게 한다.  
93         ns[i] = now_ns(CLOCK_MONOTONIC) - start;  
    → 끝 판독과의 차가 이 번의 측정값이다.  
94         printf("run%d=%lldns\n", i + 1, ns[i]);  
    → 측정값을 즉시 인쇄한다.
```

## 코드 읽기 · elapsed\_demo.c (9/9)

```
95     }  
96     → 다섯 값을 오름차순 정렬한다.  
97     qsort(ns, RUNS, sizeof ns[0], cmp_ll);  
98     printf("min=%lldns median=%lldns max=%lldns sink=%lld\n",  
99           ns[0], ns[RUNS / 2], ns[RUNS - 1], sink);  
100    → L96-L98: 정렬 뒤 첫 값=min, 가운데=median, 끝=max다.  
101    if (fflush(stdout) == EOF) {  
102        perror("stdout");  
103        return 1;  
104    }  
105    → L99-L102: flush를 확인한다.  
106    return 0;  
107    → 정상 종료 상태 0.  
108 }
```

# 3부

## 시간 기록

타임스탬프 로그로 사건을 남깁니다.

# 로그 한 줄의 설계

---

## 달력 시각

사건의 절대 시각이다.  
다른 기록과 비교할 수 있다.

## 경과 시간

프로그램 시작부터의  
흐름과 간격이 바로 보인다.

## 메시지

무엇이 일어났는지  
사람이 읽는 한 줄이다.

# tslog: 로그 한 줄 쓰기

```
static int log_line(FILE *out, const char *message)
{
    time_t now = time(NULL);
    struct tm *tm = localtime(&now);
    char stamp[32];
    strftime(stamp, sizeof stamp,
              "%Y-%m-%d %H:%M:%S %Z", tm);
    struct timespec mono;
    clock_gettime(CLOCK_MONOTONIC, &mono);
    double elapsed =
        (double)(ts_ns(&mono) - ts_ns(&start)) / 1e9;
    fprintf(out, "%s +%07.4fs %s\n",
            stamp, elapsed, message);
    return fflush(out) == EOF ? -1 : 0;
}
```

완성 예제: tslog.c

## 두 시간

strftime 달력 시각과  
MONOTONIC 경과를 남긴다.

## 한 줄 조립

fprintf 하나가 로그 한 줄이다.  
줄바꿈까지 직접 넣는다.

## 즉시 flush

fflush로 밀어 비정상 종료에도  
줄이 남게 한다.

# 로그 파일과 버퍼

```
FILE *out = fopen("run.log", "a");
if (out == NULL) {
    perror("fopen");
    return 1;
}
if (log_line(out, "start") == -1)
    return 1;

/* "a"는 파일 끝에 붙인다.
   이전 실행의 기록이 사라지지 않는다. */

if (fclose(out) == EOF) {
    perror("fclose");
    return 1;
}
```

완성 예제: tslog.c의 -o 옵션

## 추가 모드

"a"는 파일 끝에 붙인다.  
"w"는 매번 비운다.

## 버퍼링

stdio는 모아서 쓴다.  
로그는 줄마다 fflush한다.

## 닫기

fclose도 실패할 수 있다.  
반환값을 확인한다.

# 로그에서 시간의 역할

---

순서: 두 사건 중 어느 것이 먼저였는지 타임스탬프로 재구성한다.

간격: 경과 시간의 차로 각 단계가 얼마나 걸렸는지 잔다.

병합: 다른 프로그램의 로그와 달력 시각으로 시선을 맞춘다.

로그는 붙이기만 한다: 고치지 않고 이어 쓴다.

로그의 가치는 나중에 원인을 찾을 때 나온다.

# 두 시간을 함께 남기는 이유

---

## 달력 시각만

다른 기록과 비교는 되지만  
짧은 간격이 잘 보이지 않는다.

## 경과 시간만

간격은 보이지만  
절대 시점을 모른다.

## 둘 다 남긴다

언제와 얼마나를 한 줄에서  
동시에 답한다.

# tslog 실행 예시

---

```
$ TZ=Asia/Seoul ./tslog --demo
2026-09-27 16:15:35 KST +00.0004s start
2026-09-27 16:15:35 KST +00.0125s read config
2026-09-27 16:15:35 KST +00.0245s process data
2026-09-27 16:15:35 KST +00.0365s write result
2026-09-27 16:15:35 KST +00.0486s done
```

## 간격 읽기

인접한 줄의 경과 차가  
단계의 지속 시간이다.

## 정렬

경과 시간은 단조 증가하므로  
줄 순서가 곧 사건 순서다.

## 형식 고정

폭을 고정해 눈으로  
줄을 맞출 수 있다.

출력 예시: 간격은 데모 설정 값입니다.

# 시간을 잘못 쓰는 사례

---

측정에 `time(2)` 쓰기: 초 단위라 짧은 작업이 0초로 쯤다.

측정에 `REALTIME` 쓰기: 도중 시각이 수정되면 음수나 튼 값이 나온다.

`tm_mon`을 1부터 세기: 3월을 고르려면 2를 넣어야 한다.

시간대 없이 시각 남기기: 나중에 KST인지 UTC인지 알 수 없다.

`fflush` 없이 남기기: 크래시 직전의 줄이 사라진다.

# 실습 3: 로그 실행과 확인

```
TZ=Asia/Seoul ./tslog --demo
./tslog -o run.log --demo
./tslog -o run.log 'second run'
cat run.log

# read config와 process data의 간격
python3 -c 'print(24.5-12.5, "ms")'
```

값을 예측한 뒤 REPORT.md에 기록합니다.

## 데모 실행

stderr 버전과 -o 파일 버전의 출력을 비교한다.

## 추가 확인

같은 파일로 다시 실행해 줄이 이어지는지 본다.

## 간격 계산

경과 시간 열로 각 단계의 지속 시간을 계산한다.

# 코드 읽기 · tslog.c (1/11)

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <time.h>
    → L1-L5: 다섯 헤더. 시간 API는 <time.h>에 있다.
6
7 static struct timespec start;
    → 프로그램 시작 판독(단조 시계)을 저장하는 전역 변수.
8
9 static long long ts_ns(const struct timespec *ts)
10 {
11     return (long long)ts->tv_sec * 1000000000LL + ts->tv_nsec;
    → L9-L11: timespec을 나노초로 환산하는 도우미. elapsed_demo와 같다.
12 }
13
14 static void spin_ns(long long ns)
    → 약속한 나노초만큼 바쁘게 기다리는 도우미 함수.
```

# 코드 읽기 · tslog.c (2/11)

```
15 {
16     struct timespec begin;
17     → 시작 판독을 담은 timespec.
18     if (clock_gettime(CLOCK_MONOTONIC, &begin) == -1) {
19         perror("clock_gettime");
20         exit(1);
21     }
22     → L17-L20: 시작 판독 실패는 즉시 종료다.
23     struct timespec now;
24     → 반복용 현재 판독.
25     do {
26         if (clock_gettime(CLOCK_MONOTONIC, &now) == -1) {
27             perror("clock_gettime");
28             exit(1);
29         }
30     } while (ts_ns(&now) - ts_ns(&begin) < ns);
31     → L22-L26: 몸통에서 현재 판독을 얻는다. 실패 시 종료다.
32     → 두 판독의 나노초 차가 목표에 닿으면 탈출한다.
```

## 코드 읽기 · tslog.c (3/11)

```
28 }
29
30 /* 로그 한 줄 = 달력 시각 + 경과 시간 + 메시지.
31    쓴 뒤 곧바로 fflush해야 비정상 종료에도 줄이 남는다. */
32    → 주석: 로그 한 줄의 구성과 즉시 flush의 이유.
32 static int log_line(FILE *out, const char *message)
33    → 로그 한 줄을 기록하는 함수. 스트림과 메시지를 받는다.
33 {
34     time_t now = time(NULL);
35     → 달력 시각 판독.
36     if (now == (time_t)-1) {
37         perror("time");
38         return -1;
39     }
40     → L35-L38: time 실패는 -1. 호출자에게 -1로 알린다.
41     struct tm *tm = localtime(&now);
42     → 지역 시간대 기준으로 분해한다.
```

# 코드 읽기 · tslog.c (4/11)

```
40     if (tm == NULL) {
41         perror("localtime");
42         return -1;
43     }
44     → L40-L43: localtime 실패는 NULL이다.
45     char stamp[32];
46     → 시각 문자열 버퍼.
47     if (strftime(stamp, sizeof stamp, "%Y-%m-%d %H:%M:%S %Z", tm) == 0) {
48         fprintf(stderr, "strftime: buffer too small\n");
49         return -1;
50     }
51     → L45-L48: 형식 변환. 반환 0은 버퍼 부족이다.
52     struct timespec mono;
53     → 경과 시간용 단조 시계 판독.
54     if (clock_gettime(CLOCK_MONOTONIC, &mono) == -1) {
55         perror("clock_gettime");
56         return -1;
57     }
```

# 코드 읽기 · tslog.c (5/11)

```

53     }
    → L50-L53: 판독 실패도 -1이다.
54     double elapsed = (double)(ts_ns(&mono) - ts_ns(&start)) / 1e9;
    → 시작 시점과의 나노초 차를 1e9로 나눠 초로 만든다.
55     if (fprintf(out, "%s +%07.4fs %s\n", stamp, elapsed, message) < 0)
56         return -1;
    → L55-L56: 로그 한 줄을 fprintf로 조립한다. 쓰기 실패도 -1이다.
57     if (fflush(out) == EOF)
58         return -1;
    → L57-L58: 즉시 fflush. 비정상 종료에도 줄이 남는다.
59     return 0;
    → 성공 0.
60 }
61
62 int main(int argc, char **argv)
    → main. 옵션 해석, 메시지 결정, 기록의 순서다.
63 {

```

# 코드 읽기 · tslog.c (6/11)

```
64     FILE *out = stderr;
    → 기본 출력은 stderr다. 화면 로그는 표준 오류로 남긴다.
65     int demo = 0;
    → --demo 여부.
66
67     int i = 1;
    → 인자 읽기 시작 위치.
68     while (i < argc && argv[i][0] == '-' && argv[i][1] != '\0') {
    → 대시로 시작하는 옵션을 앞에서부터 읽는 반복문.
69         if (strcmp(argv[i], "-o") == 0) {
    → -o 옵션 처리 시작.
70             if (i + 1 >= argc || out != stderr) {
71                 fprintf(stderr, "usage: %s [-o FILE] [--demo | MESSAGE...]\n",
72                     argv[0]);
73                 return 2;
74             }
    → L70-L74: 파일 이름이 없거나 -o가 중복이면 usage 후 종료 2.
```

# 코드 읽기 · tslog.c (7/11)

```
75         out = fopen(argv[i + 1], "a");  
→ 추가 모드로 연다. 이전 실행의 기록이 남는다.  
76         if (out == NULL) {  
77             fprintf(stderr, "%s: %s\n", argv[i + 1], strerror(errno));  
78             return 1;  
79         }  
→ L76-L79: fopen 실패는 NULL. 경로와 errno를 남기고 종료 1.  
80         i += 2;  
→ -o와 파일 이름 두 칸을 소비한다.  
81     } else if (strcmp(argv[i], "--demo") == 0) {  
82         demo = 1;  
83         ++i;  
→ L81-L83: --demo 플래그를 기록하고 한 칸 전진한다.  
84     } else {  
85         break;  
86     }  
→ L84-L86: 모르는 대시 인자면 해석을 멈춘다. 이후는 메시지다.
```

# 코드 읽기 · tslog.c (8/11)

```
87     }
88     static char *demo_messages[] = {"start", "read config", "process data",
89                                     "write result", "done"};
→ L88-L89: 데모가 남길 다섯 메시지.
90     char **messages;
91     int nmsg = argc - i;
→ L90-L91: 메시지 배열 포인터와 개수.
92     if (demo) {
→ 데모 모드 분기.
93         if (nmsg != 0) {
94             fprintf(stderr, "usage: %s [-o FILE] [--demo | MESSAGE...]\n", argv[0]);
95             if (out != stderr && fclose(out) == EOF)
96                 perror("fclose");
97             return 2;
98         }
→ L93-L98: 데모와 메시지가 함께 오면 usage. 열린 파일은 닫는다.
99         messages = demo_messages;
```

# 코드 읽기 · tslog.c (9/11)

```
100     nmsg = 5;
    → L99-L100: 데모 메시지로 교체하고 개수를 5로 정한다.
101     } else {
    → 데모가 아니면 남은 인자가 곧 메시지다.
102     messages = argv + i;
103     if (nmsg == 0) {
104         fprintf(stderr, "usage: %s [-o FILE] [--demo | MESSAGE...]\n", argv[0]);
105         if (out != stderr && fclose(out) == EOF)
106             perror("fclose");
107         return 2;
108     }
    → L102-L108: 메시지가 없으면 usage 후 종료 2.
109 }
110
111 if (clock_gettime(CLOCK_MONOTONIC, &start) == -1) {
112     perror("clock_gettime");
113     return 1;
```

# 코드 읽기 · tslog.c (10/11)

```
114     }
115     → L111-L114: 시작 판독을 기록한다. 경과 시간 0의 기준점이다.
116     int status = 0;
117     → 전체 실패 여부를 모으는 변수.
118     for (int m = 0; m < nmsg; ++m) {
119     → 메시지별 기록 반복.
120         if (log_line(out, messages[m]) == -1) {
121             fprintf(stderr, "log write failed\n");
122             status = 1;
123             break;
124         }
125     → L117-L121: 한 줄 기록 실패면 알리고 반복을 중단한다.
126     if (m + 1 < nmsg) /* demo의 줄 사이에 약 12ms 간격을 만든다 */
127         spin_ns(12000000);
128     → L122-L123: 마지막 줄이 아니면 12ms 간격을 만든다.
129     }
130     if (out != stderr) {
131     → 파일로 남기는 경우의 정리.
```

# 코드 읽기 · tslog.c (11/11)

```
126         if (fclose(out) == EOF) {  
127             perror("fclose");  
128             status = 1;  
129         }  
→ L126-L129: fclose도 실패할 수 있다. 상태에 반영한다.  
130     } else if (fflush(stderr) == EOF) {  
131         perror("stderr");  
132         status = 1;  
133     }  
→ L130-L133: stderr 출력인 경우 flush를 확인한다.  
134     return status;  
→ 모은 실패 여부를 종료 상태로 돌려준다.  
135 }
```

## 4부

# 시계 선택과 함정

목적에 따라 시계를 고르고 남은 함정을 확인합니다.

# 벽시계가 거꾸로 가는 경우

---

NTP가 시각을 과거 방향으로 보정할 수 있다: REALTIME 값이 줄어든다.

관리자가 직접 시각을 고치면 같은 일이 생긴다.

가상 머신의 일시정지·이동도 벽시계를 점프시킨다.

그래서 간격 측정에는 MONOTONIC을 쓴다.

달력 시각 표시에는 여전히 REALTIME이 필요하다.

# 정밀도와 단위 선택

---

## 초

사람이 읽는 기록·만료 판정에  
충분한 단위다.

## 밀리초

작업 지속 시간·타임아웃의  
일반적인 단위다.

## 나노초

구간 측정의 계산 단위다.  
표시할 때 환산한다.

# clock\_show: 시계별 값과 해상도

```
static int show(const char *name, clockid_t clock)
{
    struct timespec now, res;
    if (clock_gettime(clock, &now) == -1) {
        printf("%-24s unsupported\n", name);
        return 0;          /* 없는 시계는 표시만 */
    }
    if (clock_getres(clock, &res) == -1) {
        perror("clock_getres");
        return 1;
    }
    printf("%-24s now=%lld.%09ld res=%ld\n",
           name, (long long)now.tv_sec,
           now.tv_nsec, res.tv_nsec);
    return 0;
}
```

완성 예제: clock\_show.c

## 시계 목록

REALTIME, MONOTONIC,  
RAW, CPU 시간 등을 돈다.

## 미지원 처리

없는 시계는 실패로 알려 온다.  
표시하고 지나간다.

## 해상도

res가 그 시계의  
최소 눈금이다.

# 달력 시간과 경과 시간의 역할 분담

---

## 달력 시간

언제: 로그 시각, 만료 판정,  
기록 정렬에 쓴다.

## 경과 시간

얼마나: 구간 측정, 타임아웃,  
간격 비교에 쓴다.

## 선택 기준

보여 주고 비교할 때는 달력,  
젤 때는 단조 시계다.

# 실습 4: 시계 비교 관찰

```
./clock_show
./elapsed_demo --clocks
date +%s

# 시간대 확인
TZ=Asia/Seoul date; TZ=UTC date

# (Linux 선택) 부팅 후 시간 규모 비교
cat /proc/uptime
```

## now 단위

REALTIME now는 date +%s와 같은 에포크 초다.

## MONOTONIC 기준

부팅 이후 초다.  
/proc/uptime과 규모가 비슷하다.

## res 비교

시계마다 눈금이 다른 것을  
표로 정리한다.

값을 예측한 뒤 REPORT.md에 기록합니다.

# 코드 읽기 · clock\_show.c (1/5)

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <time.h>
   → L1-L4: strerror과 시간 함수를 위한 헤더.
5
6 /* 시계마다 값의 기준과 해상도가 다르다.
7    clock_gettime이 실패하는 시계는 지원하지 않음으로 표시한다. */
   → 주석: 시계마다 기준과 해상도가 다를 수 있다.
8 static int show(const char *name, clockid_t clock)
   → 시계 하나를 조사해 한 줄로 출력하는 함수.
9 {
10     struct timespec now, res;
   → 현재값과 해상도를 담는 timespec 두 개.
```

## 코드 읽기 · clock\_show.c (2/5)

```
11     if (clock_gettime(clock, &now) == -1) {
12         printf("%-24s unsupported (%s)\n", name, strerror(errno));
13         return 0;
→ L11-L13: 조회 실패는 그 시계가 없다는 뜻. 미지원 표시 후 계속한다.
14     }
15     if (clock_getres(clock, &res) == -1) {
16         perror("clock_getres");
17         return 1;
18     }
→ L15-L18: 해상도 조회 실패는 치명적으로 보고 종료 1.
19     printf("%-24s now=%lld.%09ld res=%ld.%09ld\n", name,
20           (long long)now.tv_sec, now.tv_nsec,
21           (long)res.tv_sec, res.tv_nsec);
→ L19-L21: 이름·now·res를 한 줄로 출력한다. %-24s가 열을 맞춘다.
```

## 코드 읽기 · clock\_show.c (3/5)

```
22     return 0;
    → 성공 0.
23 }
24
25 int main(int argc, char **argv)
    → main. 여섯 시계를 차례로 조사한다.
26 {
27     if (argc != 1) {
28         fprintf(stderr, "usage: %s\n", argv[0]);
29         return 2;
30     }
    → L27-L30: 인자 오류는 usage와 종료 상태 2.
31     int status = 0;
    → 시계별 실패를 모으는 변수.
```

## 코드 읽기 · clock\_show.c (4/5)

```
32     status |= show("CLOCK_REALTIME", CLOCK_REALTIME);  
    → 벽시계, 달력 시각의 기준이다.  
33     status |= show("CLOCK_MONOTONIC", CLOCK_MONOTONIC);  
    → 단조 시계, 부팅 후 흐름이다. 구간 측정에 쓴다.  
34 #ifdef CLOCK_MONOTONIC_RAW  
35     status |= show("CLOCK_MONOTONIC_RAW", CLOCK_MONOTONIC_RAW);  
36 #endif  
    → L34-L36: RAW 시계는 표준이 아니어서 매크로 확인 후 컴파일한다.  
37 #ifdef CLOCK_BOOTTIME  
38     status |= show("CLOCK_BOOTTIME", CLOCK_BOOTTIME);  
39 #endif  
    → L37-L39: BOOTTIME은 Linux 확장. 절전 시간을 포함한 흐름이다.  
40 #ifdef CLOCK_PROCESS_CPUTIME_ID  
41     status |= show("CLOCK_PROCESS_CPUTIME_ID", CLOCK_PROCESS_CPUTIME_ID);
```

# 코드 읽기 · clock\_show.c (5/5)

```
42 #endif
    → L40-L42: 이 프로세스가 쓴 CPU 시간을 쟀다.
43 #ifdef CLOCK_THREAD_CPUTIME_ID
44     status |= show("CLOCK_THREAD_CPUTIME_ID", CLOCK_THREAD_CPUTIME_ID);
45 #endif
    → L43-L45: 이 스레드가 쓴 CPU 시간을 쟀다.
46     if (fflush(stdout) == EOF) {
47         perror("stdout");
48         return 1;
49     }
    → L46-L49: flush를 확인한다.
50     return status;
    → 모든 실패 여부를 종료 상태로 돌려준다.
51 }
```

# 확인 퀴즈

---

경과 시간을 잴 때 `CLOCK_MONOTONIC`을 쓰는 이유는?

`time_t`에서 화면용 문자열까지 만드는 호출 순서는?

`struct tm`의 `tm_mon`이 9라면 몇 월인가?

`strftime`의 반환값이 0이면 무엇을 뜻하나?

로그 한 줄에 달력 시각과 경과 시간을 함께 남기는 이유는?

# 실습 정리와 제출

---

calendar\_demo의 local·utc를 TZ 실험과 date 출력과 맞춘다.

tm\_normalize의 네 사례 결과와 이유를 적는다.

elapsed\_demo의 min·median과 두 시계의 now·res를 기록한다.

tslog 로그를 붙여 넣고 단계 간격을 계산한다.

C 소스, REPORT.md, 실행 로그를 제출한다.

선택 확장: tslog에 단위 옵션 추가

## 참고 자료

---

Linux man-pages: `time(2)`, `difftime(3)`, `clock_gettime(2)`

Linux man-pages: `ctime(3)`, `strftime(3)`, `tzset(3)`, `fopen(3)`

Linux man-pages: `times(2)`, `proc(5)`, `date(1)`

다음 주에는 프로세스 정보와 `times(2) · CLK_TCK`로 이어집니다.

수업은 Ubuntu 일반 사용자 환경을 기준으로 합니다.